



SENG201

FUNDAMENTALS OF

SOFTWARE ENGINEERING

Lecture Slides

LECTURER:
Wumi AJAYI, Ph.D

Department of
Software Engineering,
Babcock University.

Ajayi, Olusoga, Ayite, Akande, Adetunji

TOPICS

- 1. Course Content/Introduction**
- 2. Software Development Life Cycle & Process Models**
- 3. Managing and Planning Software Projects**
- 4. Proposal Writing**
- 5. Introduction to Risk**
- 6. Requirements Engineering**
- 7. Modelling Requirements-** Introduction to UML; use case modelling and Scenario Analysis; UML Sequence diagram for key use cases; State diagram.
- 8. Design models**
- 9. Usability Engineering**
- 10. Software Testing**
- 11. Project Monitoring and Quality Assurance -** Configuration Management; Using Earned Value Analysis for monitoring progress

WEEK ONE

Course Content / Introduction

This week's lecture aims to cover the following:

- *Course Expectations*
- *Lecture Topics*
- *Class Conduct*
- *Course Grading*
- *Characterizing a Software*
- *Defining and Appreciating Software Engineering*
- *The Software Engineering Profession*
- *Software Engineering Ethics*

Course Content/Introduction

Lecture Objectives

At the end of this week's lecture, students are expected to:

1. Know the course content, course grading and the expectations of SENG201 as a course
2. Understand what software engineering is all about
3. Appreciate the need for software engineering and the software engineering profession
4. Know the ethics that govern a software engineer

Course Content

Course Description

SENG201: Fundamentals of Software Engineering

- The course covers the basics of the Software Engineering global practice. It is aimed at helping students appreciate Software Engineering concepts and terminologies for large and complex systems.
- As a result we will cover topics associated with appropriate processes involved in the production of quality software products.

Course Content

What should you expect from INSY405?

- This course is designed to expose students to the methods used to deal with issues encountered during the design and implementation of software systems.
- Typical Class sessions will involve the use of clearly described software project scenarios to help understand the various software engineering methods and their application.
- Students will be required to work in groups while they attempt practice exercises during practical classes.
- **Students are also made to understand that there are many ways to solve a software problem, but Software Engineering aims to apply the method that produces the best quality using minimal resource**

Course Content

Course Activities

1. Lectures
 2. Reading Assignments/Class Interactions
 3. Class Teams
 4. Team Assignment
 5. Case Studies
 6. Group Interactions
-

Class Conduct

1. Lateness to class will not be tolerated. No student will be allowed entry after 5 minutes
2. Students are encouraged to contribute to class discussions. Bonus marks will be awarded discretionally
3. Students are expected to display high level of discipline in conduct and speech

Course Content

Recommended Text

1. Fundamentals of Software Engineering (2nd Edition) by Carlo Ghezzi, Mehdi Jazayeri and Dino Mandrioli, 2002
2. Object-Oriented Software Engineering: Practical Software Development using UML and Java
3. Software Engineering: Theory and Practice (4th Edition) by Shari Lawrence Pfleeger & Joanne M. Atlee, Pearson Education, 2010
4. Software Engineering (7th edition) by Ian Sommerville, Addison Wesley, 2004.
5. Applying UML and Patterns – An Introduction to Object-Oriented Analysis and Design and the Unified Process (2nd Edition) by Craig Larman

Course Content

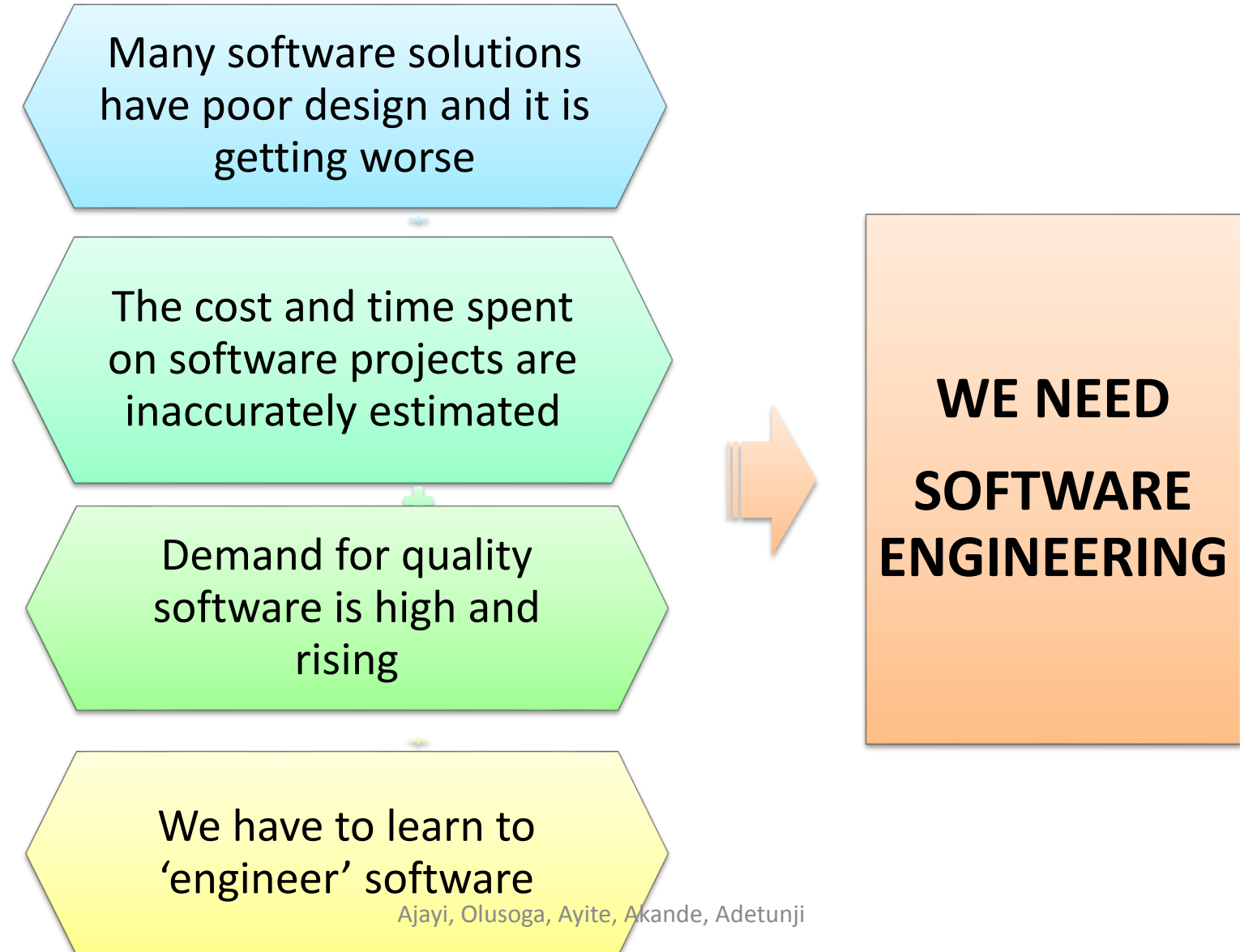
Course Grading

The grading structure for this course is itemized in the table below:

Grading Component	Weight	Remarks
Attendance	5 Marks	This also includes attendance at practical class
Quizzes	10 Marks	Quizzes in class will be un-announced. Students are encouraged to revise previous lecture(s) and go through next lecture. Be prepared always! Quizzes may be given in practical class too
Assignment	10 Marks	Details will be communicated in practical class
Mid-Semester	15 Marks	A mid-semester examination will be carried out, during the period stipulated by the University
Group Project	20 Marks	Details will be communicated in practical class
Final Examination	40 Marks	Examinations will be based of case studies. Students are advised to get a clear understanding of the concepts taught in the course.
TOTAL	100 MARKS	

Introduction

Why Software Engineering?



Introduction

What is a Software?



- Software can be difficult to describe because it is not tangible!
- The term Software, is a generic term for an organized collection of computer data and programming instructions
- Software is created using a programming language and related utilities. These utilities may be script interpreters, compilers, IDEs (Integrated Development Environments), etc.

Reading Assignment:

Is there any difference between a software and a program?

Ajayi, Olusoga, Ayite, Akande, Adetunji

Introduction

About Software

Easy to modify

No “wear and tear” - unlike hardware

Unskilled people can create one

Difficult to see development effort

Easy to reproduce / replicate

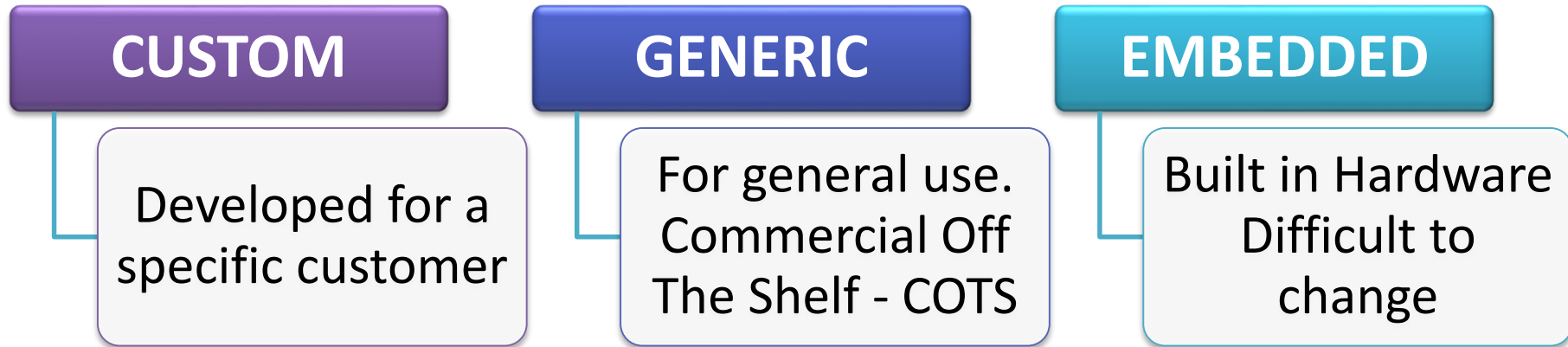
The Unique Nature of Software

Ajayi, Olusoga, Ayite, Akande, Adetunji

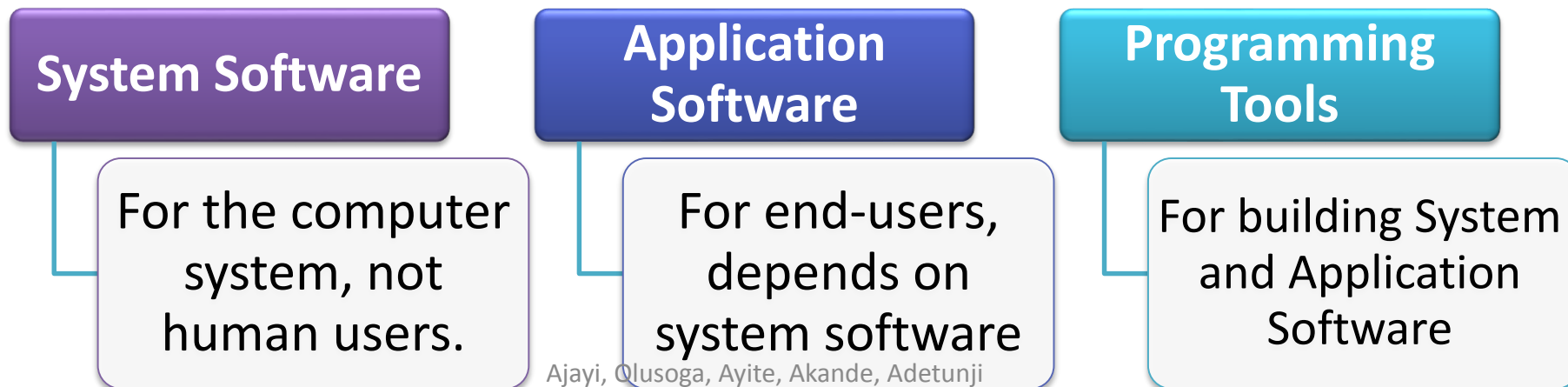
Introduction

About Software

Types of Software



Software Categories



Introduction

What is Software Engineering?

The process of solving customers' problems by the systematic development and evolution of large, high-quality software systems within cost, time and other constraints

Introduction

What is Software Engineering?

IEEE:

- (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, maintenance of software; that is, the application of engineering to software.
- (2) The study of approaches as in (1).

The Canadian Standards Association: The systematic activities involved in the design, implementation and testing of software to optimize its production and support.

The establishment and use of sound engineering principles in order to economically obtain software that is reliable and works efficiently on real machines

Introduction

Software Engineering Explained

Solving customers' problems:

- This is the main goal of software engineering
- Sometimes the solution is to buy, not build
- Adding unnecessary features does not help solve the problem
- Software engineers must communicate effectively to identify and understand the problem



Introduction

Software Engineering Explained

Systematic Development and Evolution:

- An engineering process involves applying *well understood techniques* in an organized and *disciplined* way
- Most development work are mainly an *evolution* from a previous scheme of work



Introduction

Software Engineering Explained

Large, High Quality Software System:

We are dealing with large systems – they *cannot be completely understood* by one person



**We need Teamwork
and co-ordination**



**How do we divide
the work and still
ensure delivery**



**End product
MUST be
Satisfactory**

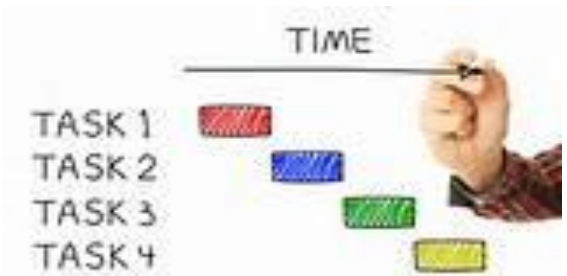
Introduction

Software Engineering Explained

Cost, Time and Other Constraints:

Resources are limited – Money, Time, etc.
Therefore,

1. The **benefit** must outweigh the **cost**
2. An **accurate estimate** of cost and time must be made
3. Remember that others are **competing** to do the job cheaper and faster



Introduction

Is there a demand for Software Engineers?

Home > Careers > Jobs

INFOWORLD TECH WATCH
By **Ted Samson** | Follow

About | 
Informed news analysis every weekday







Demand for software engineers keeps climbing -- and so do the salaries

A hot job market for software engineers is driving up salaries, with the biggest paychecks earned in California

MORE LIKE THIS
2013 IT salaries: 15 titles getting the biggest pay raises

How attractive is the Software Engineering Profession?

In terms of average base salary, job availability and career progression, the software engineering profession is 2nd on the list in America.

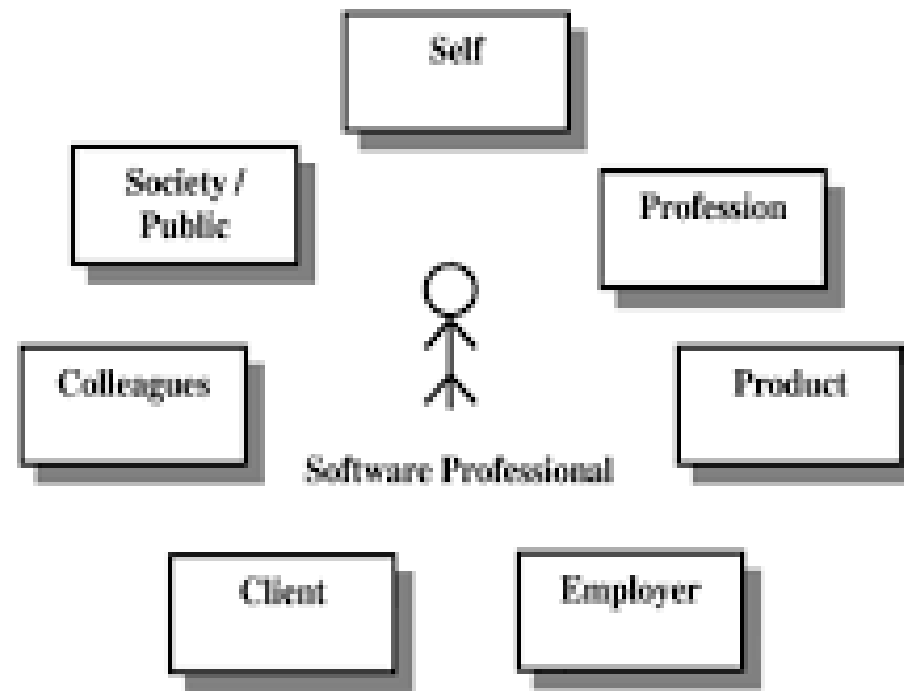
This information was gotten from [25 Best Jobs in America 2015](#), a survey conducted by career, community and employment website Glassdoor (Adams, 2015)

Introduction

Ethics that guide the Software Engineering Profession

Software engineering ethical conduct requires software engineers to always:

1. Act consistently with **public interest**
2. Act in the **best interests** of their **clients**
3. Develop and maintain with the **highest standards** possible
4. Maintain **integrity** in professional judgment
5. Encourage an ethical approach in management
6. Advance the integrity and **reputation of the profession**
7. Be **fair and supportive** to colleagues
8. Participate in **lifelong learning**



WEEK TWO

Software Development Life Cycle and Process Models

This week's lecture aims to cover the following:

- *SDLC*
- *Software Development Process Models*
- *Knowing how to choose the most appropriate model*
- *Class Examples/Discussions*

Software Development Life Cycle and Process Models

Lecture Objectives

At the end of this week's lecture, students are expected to:

1. Have a clear understanding of the SDLC concept
2. Know all the SDLC stages, in the “right order”
3. Know the connection between the SDLC and the different process models
4. Understand the different software development process models, so as to know the most applicable model for different software project scenarios.

Software Development Life Cycle and Process Models

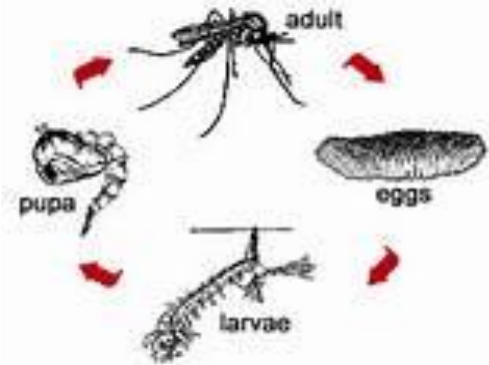
Software Development Life Cycle (SDLC)

Software Development has a Life Cycle???



- When thinking about SDLC, think about phases and forms.
- The merriam webster dictionary defines the term 'life cycle' as "the series of stages in form and functional activity through which an organism passes between successive recurrences of a specified primary stage"

E.g. The life cycle of a mosquito is from egg to larva to pupa then to adult



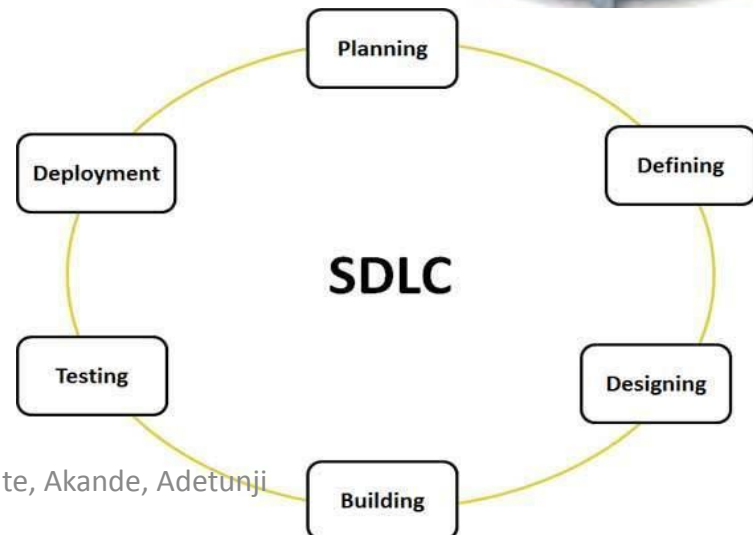
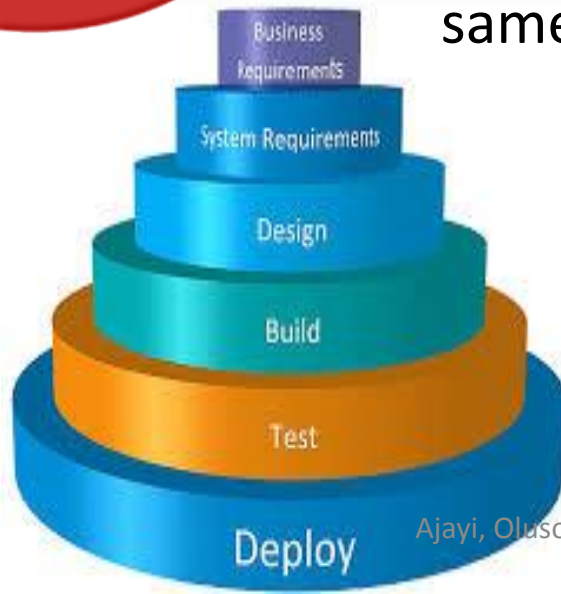
- SDLC has to do with the different phases (and forms) that a software under development undergoes, in order to have a complete deployable software product.

Software Development Life Cycle and Process Models

Software Development Life Cycle (SDLC)



Various diagrams and phrases describe the different stages of software development, but they all mean the same in essence



Software Development Life Cycle and Process Models

Software Development Life Cycle (SDLC)

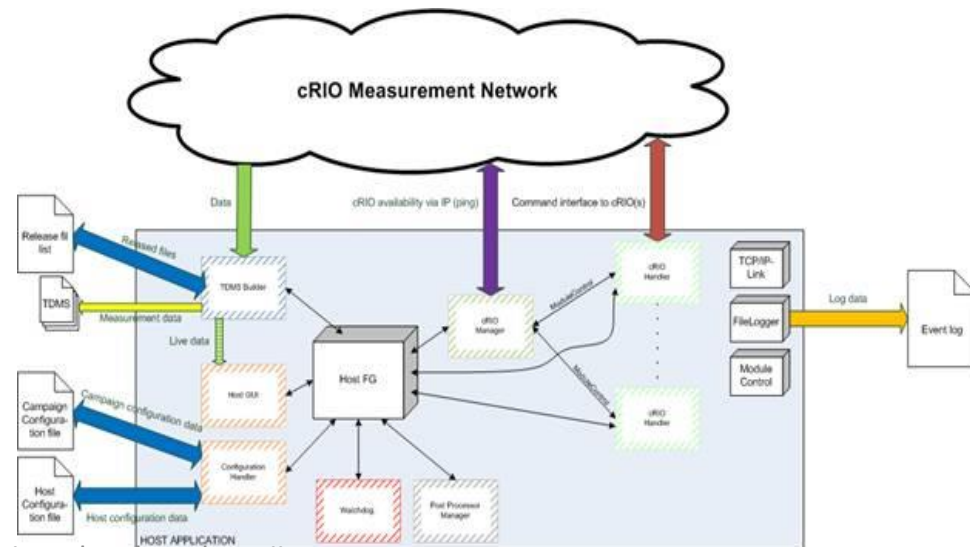
➤ **Requirements Elicitation & Analysis:**

This phase identifies business requirements. Requirements such as: “Expected Users of the System” and what the system is expected to do, are determined and analysed. This phase produces a Requirements Specification document



 Design:

The system and software design is produced using details derived from the specification document

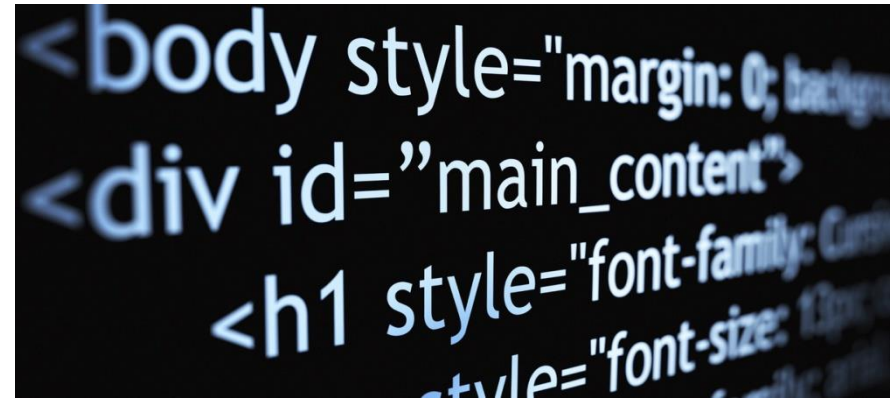


Software Development Life Cycle and Process Models

Software Development Life Cycle (SDLC)

➤ **Implementation (or Coding):**

The designed blueprint is turned to reality by writing the source code of the entire application



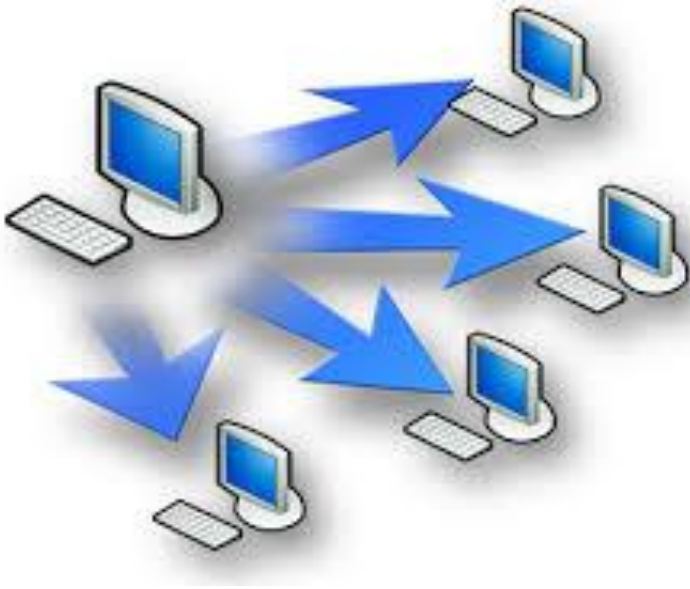
➤ **Testing:**

The software is tested against the initially identified requirements. This is to ensure that the product is truly solving the needs gathered during the requirements phase



Software Development Life Cycle and Process Models

Software Development Life Cycle (SDLC)



➤ **Deployment:**

The software is installed on systems. Client's custom software are deployed at client's site, while generic software are made available via appropriate medium. Once the product is tested and ready to be deployed it is released formally in the appropriate market



➤ **Support & Maintenance:**

The software team is also responsible for support and maintenance. Plans on how to scale and enhance your system are made

Software Development Life Cycle and Process Models

Software Development Life Cycle (SDLC)

*Who
does
What
?*

SDLC VS SOFTWARE DEVELOPMENT ROLES

Most Software Engineering Organisations structure workflow, team formation and project dependence differently.

This difference is mainly as a result of varying business strategies and priorities.

Analysts
Business Analysts
Solutions Architects
Testers
Requirement Engineers
Programmers
Support Team
Developers

Software Development Life Cycle and Process Models

Software Development Process Models

Now that we know the different stages of software development,
How then do we go about the development of the software?



This leads us to the different Software development processes

Ajayi, Olusoga, Ayo, Ayanade, Adetunji

Software Development Life Cycle and Process Models

Software Development Process Models

Software development process models describe the different approaches that can be employed in the creation of a software solution.

- They are the approaches and techniques which define the foundation pillar of the software development life cycle
- As the software engineering profession grows, more development strategies evolve.

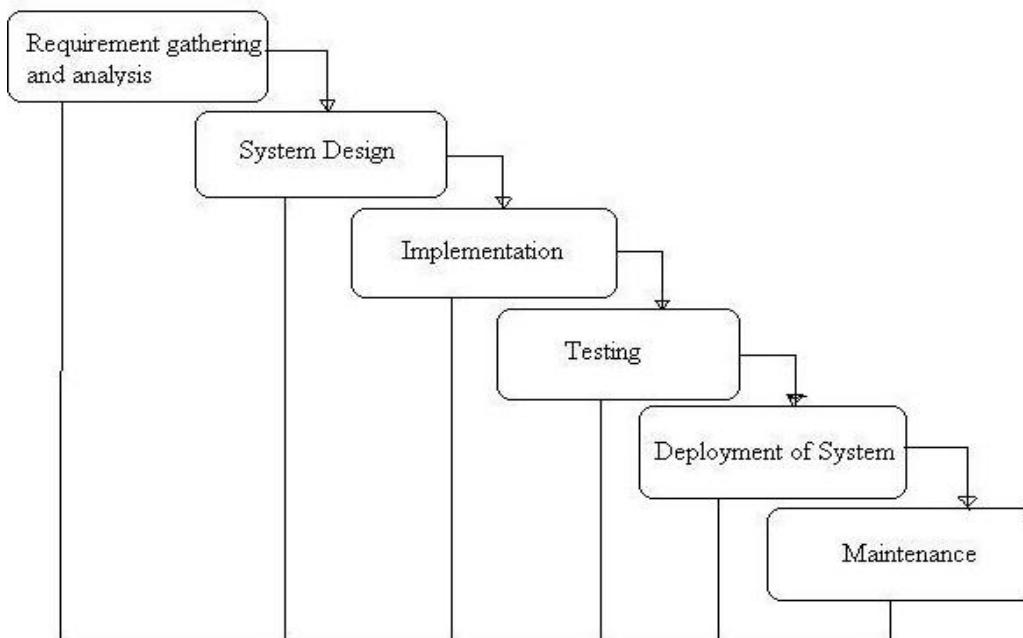
Some traditional process models are:

- Waterfall
- Iterative
- Incremental
- Spiral
- Big Bang
- Prototyping
- Re-use based
- Agile

Process Models

Waterfall Model

General Overview of "Waterfall Model"



- The waterfall model is the traditional model of software development
- It follows a serial execution of the SDLC phases
- The waterfall model upholds the believe that each phase should be given adequate attention and should be completed before proceeding to the next phase of the SDLC
- Each phase has specific deliverables and a review process

Advantages

- Simple to understand and use
- Easy to manage
- Highly systematic in nature
- Enables proper documentation

Disadvantages

- Hard to get ALL requirements at project start
- Difficult to go back to a previous phase
- High risk and uncertainty - no risk analysis
- Working software available late in the project
- Requires heavy documentation
- Developers are often delayed between phases

Process Models

Big Bang Model



- This process model is based on the theory of the creation of the universe
- Money and people are gathered to build a software, within flexible deadline
- It is useful for small projects, and often results into repetitive development
- It requires little or no formal testing

Advantages

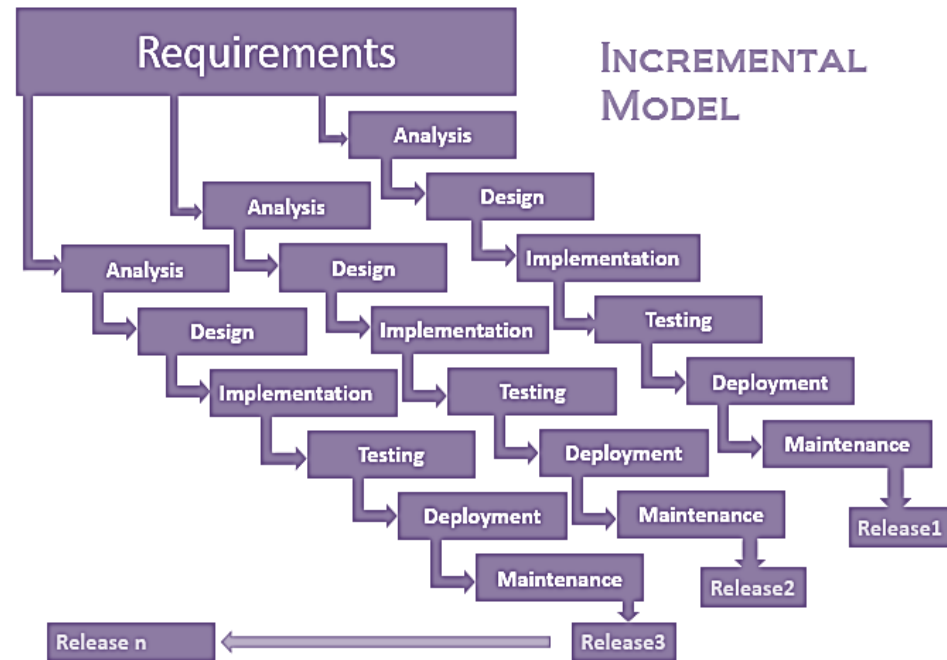
- Useful when requirements are not very clear
- A very simple and easy model with no rules. Hence easy for starters
- Requires less resource
- Gives room for developers to be creative and flexible

Disadvantages

- Known to have high product failure rate
- High level of uncertainty – very risky
- Cannot be used for Large, complex systems
- Does not fit into object oriented design and development
- Gives little or no room for thorough requirements analysis

Process Models

Incremental Model



- This process model breaks a development project/product into divisions called **modules** for easy management
- Each module is developed separately
- The 1st module is often a major aspect of the system that can be used, and other module integrated incrementally
- It can be seen as a “multiple-waterfall” process

Advantages

- A working version of software is produced after the first module
- High success rate
- Easy to manage and monitor progress
- Reduced risk of requirements changes

Disadvantages

- It can sometimes be difficult to agree on which increment is the last increment
- Problems may arise from additional increments.

Process Models

Iterative Model



- This process model is used to develop a system in repetitions (iterative)
- Each iteration gives rise to an executable release
- With each iteration, extra features can be designed, developed and tested until there is a fully functional software (ready for deployment)

Is there any difference between incremental and iterative development?

Advantages

- Early and periodic results
- Easy to measure progress
- Requirements change are less costly
- Easier to manage risk - High risk part is done first

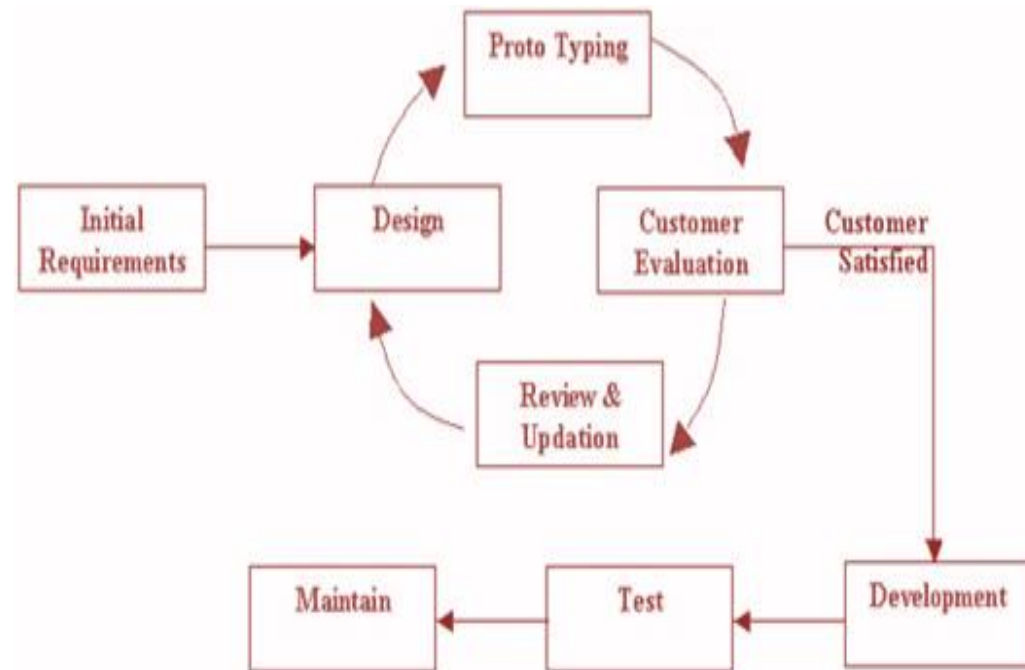
Disadvantages

- Difficult to know when to end project
- Design issues may occur since requirements are not concluded at the beginning
- Not suitable for smaller projects
- Intense customer involvement

Process Models

Prototyping

- Prototyping is the act of developing a sample version of an intended software solution.
- It allows users evaluate developer proposals before implementation
- It also helps understand user specific requirements



Advantages

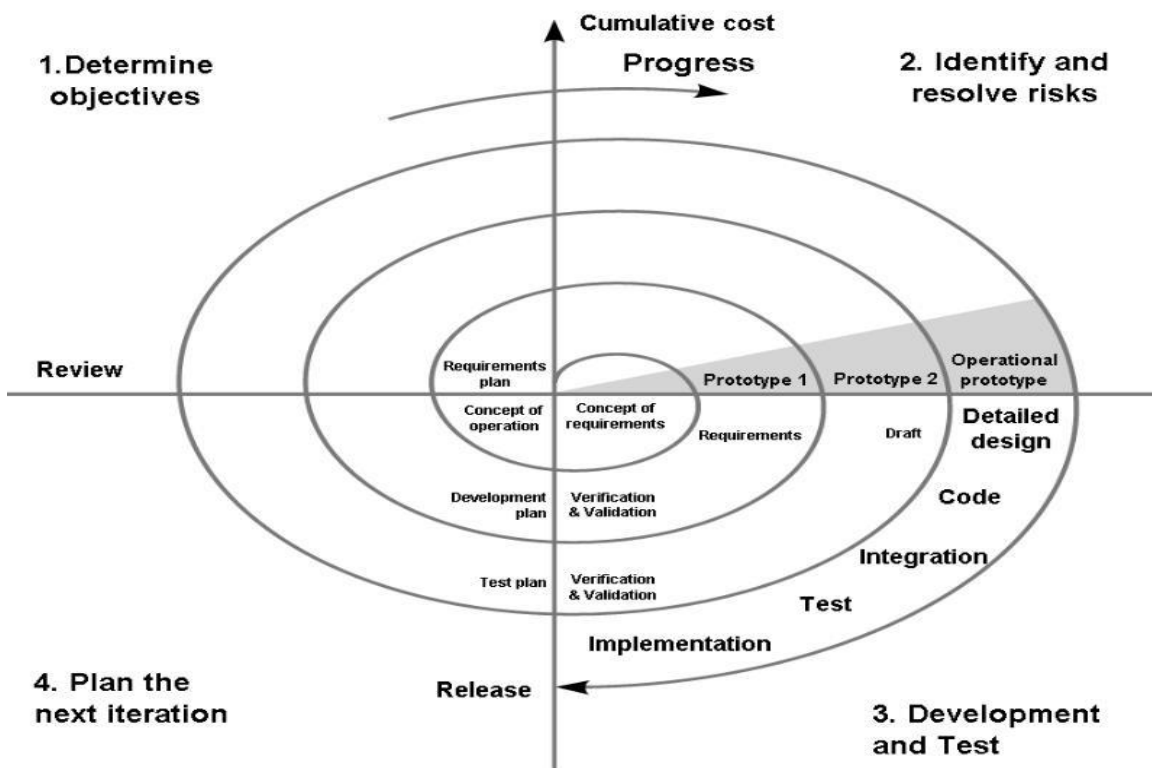
- It provides reality-check on initial project plan and estimates
- Implementation is based on a tried sample
- Early feedback
- High user involvement
- Easy to identify non-captured requirements

Disadvantages

- Customer satisfaction may take a while
- High risk of compromising quality
- Building prototypes may be too much if not monitored properly

Process Models

Spiral Model



- The spiral method is a risk-biased development strategy
- It consciously iterates risk analysis at a predefined frequency
- It is a combination of iterative development process model and waterfall model
- It analyses alternatives and weighs their viability

Advantages

- A good model for very large products
- Risk analysis hints on required testing modalities
- Working software is produced early
- Accommodates changing requirements

Disadvantages

- Spiral may go on longer than planned
- Not suitable for smaller projects – expensive
- Expert skills needed for risk analyses

Spiral Model



Process Models

Agile

- The agile model is perhaps the most widely used model
- It is mostly used in time-critical situations
- It uses iterative development as a basis, but its centred on people's feedbacks instead of using planning as the primary control mechanism
- There are many variations of agile model - Agile Modeling, Extreme Programming, Feature Driven Development, etc.

Advantages

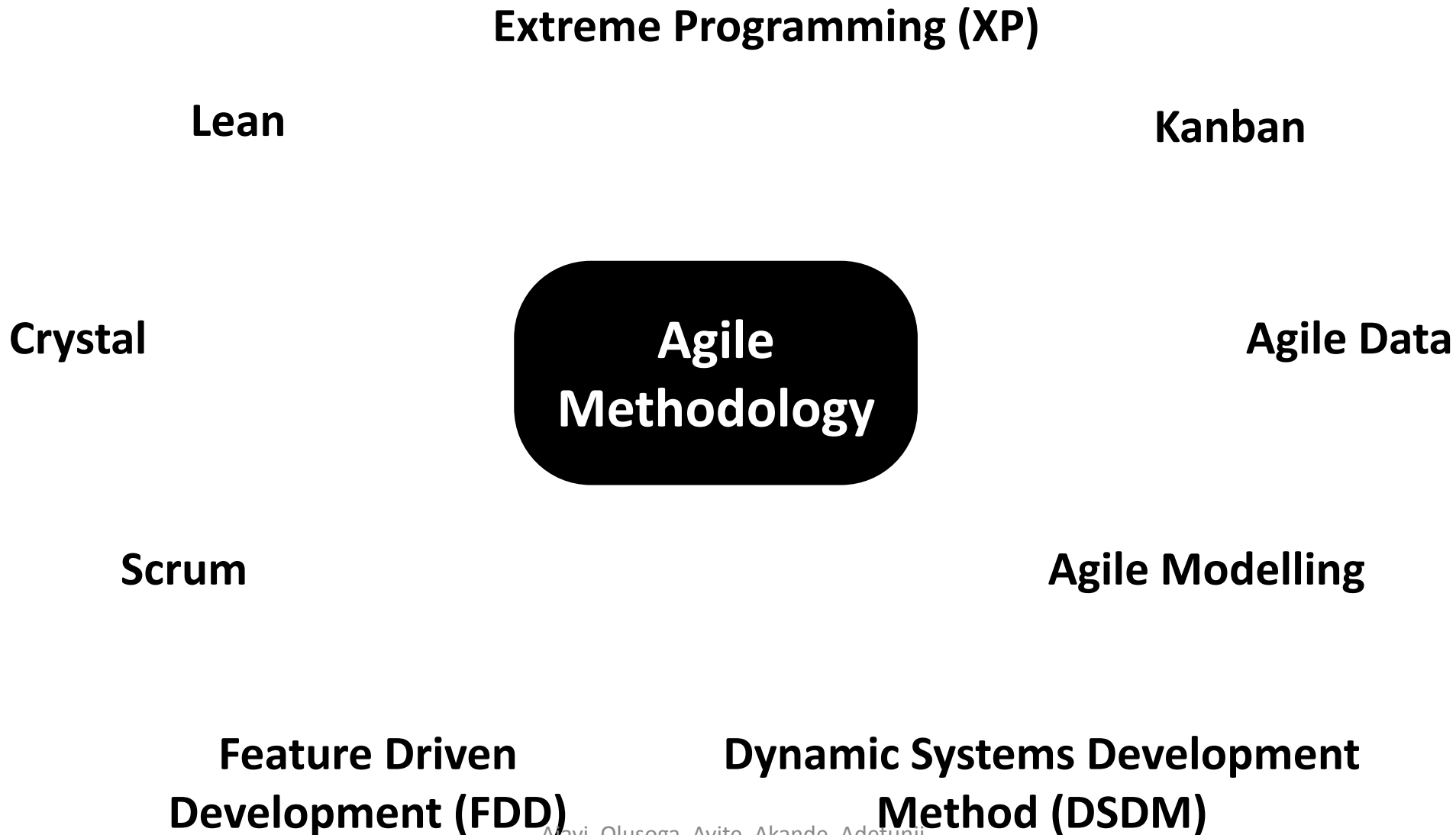
- Minimal development rules
- Documentations easily employed
- Customer satisfaction by rapid, continuous delivery of useful software

Disadvantages

- Poor documentation
- Very high project risk
- Poor emphasis on design principles

Process Models

Present Day Variants of the Agile Methodology



Process Models

Variants of the Agile Methodology

XP	FDD	Scrum
A code first approach that focuses on having results in increments using continual testing and revision, in order to achieve customer satisfaction.	An adaptive development organized around 5 process: 1. Develop an overall model 2. Build features list 3. Plan by feature 4. Design by feature 5. Build by feature Features serve as a basis for team formation	A software development framework that focuses on proper management and adequate control. It aims to deliver a working software incrementally and empirically. The framework is self-organizing, cross-functional in team creation
It allows developers to respond to changing requirements – through constant communication It denounces the advantage in thorough and time-consuming analysis. Hence no need for specialization of duties, all team members write codes, test, design and evaluate.	A Feature in FDD refers to as a small, client-valued function expressed in the form <action> the <result> <by for of to > <a(n)> <object> . A Feature Set is a group of business related features. The planning team reviews the features set assigned to programmers An agile methodology for large projects and teams	It is useful for complex projects. There is no overall team leader the decision is made by the team at different stages of the development Product Backlog ———— Wish list Sprint planning ———— Pull small chunk Sprint ———— Code in Short time ScrumMaster – keeps the team focused

Software Development Life Cycle and Process Models

Choosing Process Models

- It can be very tricky deciding on what process model to follow.
- Two or more process models may be used in carrying out a software project, but the level of success may vary
- A deep consideration of the most appropriate model is required of a software engineering team.
- The following factors may be given deep consideration when deciding on what model to use

Requirements
Stability

Team
Dynamics

Time
Constraint

Domain
Familiarity

Consequence
of wrong
analysis

Customer
Involvement /
Availability

Developers'
Proficiency

Financial
Constraint

Familiarity
with
Technology

Geographical
Dynamics

Ajayi, Olusoga, Ayite, Akande, Adetunji

Read further: <https://melsatar.wordpress.com/2012/03/21/choosing-the-right-software-development-life-cycle-model/>

Software Development Life Cycle and Process Models

Choosing Process Models

How can you decide on which method or process to implement while developing a software?

This depends on the feasibility analysis and key requirements factor.

- For larger projects, it is advisable to use Spiral and Incremental model. Development can be flexibly done in modules and parts.
- When dealing with small projects that have fixed requirements, you can decide to go with the waterfall model. Such projects do not require changes during implementation

The size of a project is not enough to determine the appropriate process model(s). Other factors mentioned earlier should also be considered

Software Development Life Cycle and Process Models

Reading Assignment

For all the process models, list out a clear distinction of when to use of them

Choosing Process Models – Class discussion